Current Progress Report
ESP32 Adaptive Sampling IoT Project

IoT Adaptive Sampling Project

April 18, 2026

1 Introduction

This document is the current detailed progress report for the project. It is meant to complement the older historical report in `PHASE1_REPORT.tex`, which documents Phases 1 to 7 in depth but does not reflect the latest hardware validation, saved MQTT evidence, three-signal results, or the current project status.

The purpose of this report is to present:

- what has been implemented so far,
- what has already been validated on real hardware,
- what measured results exist today,
- what is still pending before final submission.

2 Executive Summary

The project now has a working `ESP32 + FreeRTOS` pipeline that:

- generates the required input signal through a virtual sensor,
- collects samples into fixed-duration windows,
- estimates the dominant frequency from each window,
- adapts the sampling rate at runtime,
- computes a per-window aggregate,
- publishes real aggregate messages to a nearby edge server over `MQTT/WiFi`,
- publishes compact aggregate payloads through `LoRaWAN + TTN`.

The main local validation milestones are complete. Current evidence covers:

- `MQTT/WiFi` edge delivery,
- `LoRaWAN/TTN` cloud delivery,

- adaptive sampling and three signal profiles,
- anomaly-filter evaluation and communication-volume comparison,
- `INA219` energy measurement on real hardware.

The only intentionally weaker item is:

- one live `MQTTS` validation run if a TLS-capable broker is available.

3 Current Phase Status

Phase	Topic	Status	Notes
1	Environment setup	Complete	Board, PlatformIO, serial, and broker environment validated
2	Firmware skeleton	Complete	Modular FreeRTOS structure and shared data types implemented
3	Virtual sensor	Complete	Required sinusoidal signal generated in firmware
4	Maximum sampling frequency	Complete	Strict benchmark executed on the real board
5	Frequency analysis	Complete	Dominant-frequency detection validated
6	Adaptive sampling	Complete	Runtime update from `50 Hz` to `40 Hz` validated
7	Aggregate computation	Complete	Window averages computed and passed downstream
8	MQTT over WiFi	Complete	End-to-end home-network validation completed on hardware
9	LoRaWAN + TTN	Complete	Integrated main-app LoRaWAN validation succeeded on real hardware
10	Performance measurements	Mostly complete	Latency, payload-size, and INA219 energy evidence exist
11	Optional physical sensor / extras	Optional	Not required for the core project
12	Final packaging and write-up	In progress	README, evidence map, and screenshot assets are being prepared

4 Project Architecture

The implemented data path is:

```
virtual signal
    -> sampling task
    -> FFT / dominant-frequency detection
    -> adaptive sampling control
    -> 5 s aggregate window
    -> MQTT over WiFi to nearby edge listener
    -> compact LoRaWAN payload for TTN
```

The modular structure is important because each task has a clear responsibility:

- `signal_input`: sample generation and runtime sampling-rate changes,
- `signal_processing`: window buffering, frequency estimation, and aggregate creation,
- `sampling_control`: adaptive policy,
- `comm_mqtt`: edge publishing,
- `comm_lorawan`: compact uplink payload preparation,
- `metrics`: runtime counters and timing support.

5 Hardware And Software Used

5.1 Hardware

- `Heltec WiFi LoRa 32 V3` compatible board
- integrated `WiFi`
- integrated `868 MHz LoRa` radio
- integrated OLED
- laptop running the local broker and edge listener

5.2 Software

- `PlatformIO`
- `ESP-IDF`
- `Mosquitto`
- Python-based `MQTT` listener and analyzer
- `BetterSerialPlotter`

6 Signal Model And Profiles

The core virtual input signal follows the assignment requirement:

```
2*sin(2*pi*3*t) + 4*sin(2*pi*5*t)
```

This produces two spectral components:

- `3 Hz`
- `5 Hz`

The current implementation uses three profiles:

Profile	Purpose	Notes
`clean_reference`	baseline	deterministic signal without noise
`noisy_reference`	robustness	same signal plus low random noise
`anomaly_stress`	bonus / stress	noisy signal plus sparse spike injection

The selected profile is carried through the aggregate object and the MQTT payload as `signal_profile`, and anomaly windows also report `anomaly_count`.

7 Maximum Sampling Frequency Benchmark

The strict benchmark from Phase 4 tested multiple target rates and classified a rate as stable only if timing stayed clean under the project rule.

7.1 Measured Result Table

Target	Achieved	Drops	Miss.	Stable	Observation
`50 Hz`	`50.25 Hz`	0	0	Yes	clean stable run
`100 Hz`	`100.25 Hz`	0	1	No	very close, but one miss occurred
`200 Hz`	`200.25 Hz`	0	2	No	throughput held, timing did not
`250 Hz`	`250.25 Hz`	0	12	No	watchdog instability started
`500 Hz`	`500.25 Hz`	0	26	No	repeated instability
`1000 Hz`	`1000.00 Hz`	1	55	No	severe instability

7.2 Interpretation

The important conclusion is:

- the board can technically generate and consume data above `50 Hz`,
- but under the current strict stability rule, the highest clean operating point is `50 Hz`.

This is why the project uses:

- `50 Hz` as the baseline oversampling reference,
- `40 Hz` as the first stable adaptive rate for the current signal.

8 FFT And Adaptive Sampling

After the first stable `50 Hz` window, the firmware detects the dominant component correctly and updates the rate:

```
Window 0 ready | fs=50.0Hz samples=250 avg=0.0041 min=-5.6139 max=5.6139
    duration=4.97s
Adaptive sampling update | window=0 dominant=5.00 Hz peak=2.0000 previous_fs
    =50.0 Hz new_fs=40.0 Hz
Applying adaptive sampling frequency 40.0 Hz
```

8.1 Why `40 Hz` Is Correct

The current policy uses:

- oversampling factor: `8x`
- dominant frequency: `5 Hz`

So:

```
5 Hz * 8 = 40 Hz
```

That result matches the validated runtime behavior.

9 Aggregate Computation

Each completed window becomes an aggregate object that contains:

- `window_id`
- `window_start_us`
- `window_end_us`
- `sample_count`
- `sampling_frequency_hz`
- `dominant_frequency_hz`
- `average_value`
- `signal_profile`
- `anomaly_count`

The first clean aggregate appeared at the initial baseline rate:

```
Aggregate result | window=0 fs=50.0Hz samples=250 avg=0.0041 dominant=5.00 Hz
```

After adaptation, the aggregate pipeline remained correct:

```
Aggregate result | window=1 fs=40.0Hz samples=200 avg=-0.0000 dominant=5.00 Hz
```

This confirms that:

- the aggregate function is correct,
- the adaptive transition does not break the aggregate path,
- the edge and cloud paths can consume real summary data instead of placeholders.

10 MQTT Over WiFi Validation

The local edge path was fully validated on the real board on the home network.

10.1 What Was Proven

- the Heltec connected to the home WiFi,
- the board reached the local broker,
- the edge listener received real aggregate messages,
- the messages contained synchronized timestamps,
- the saved run had no missing windows.

10.2 Latest Clean Listener Run

Source: `results/summaries/mqtt_summary_2026-04-18_listener.md`

Overview

Metric	Value
Messages received	`5`
Unique windows	`5`
Missing windows	`0`
First window	`296`
Last window	`300`
Received span	`20.002 s`
Signal profile	`clean_reference`
Total payload bytes	`2270`

Timing Metrics

Metric	Min	Avg	P95	Max
`listener_latency_us`	1232577	1234421.6	1239997	1239997
`end_to_end_latency_us`	1584866	1587868.6	1596541	1596541
`edge_delay_us`	352289	353447.0	356544	356544

Signal Metrics

Metric	Min	Avg	Max
`sample_count`	200	200.0	200
`sampling_frequency_hz`	40.0	40.0	40.0
`dominant_frequency_hz`	5.0	5.0	5.0
`payload_size_bytes`	454	454.0	454

10.3 Interpretation

The edge path is no longer just implemented in code. It has been validated end-to-end with:

- real board,
- real WiFi,

- real broker,
- real listener output,
- saved artifacts.

```
MQTT Listener Live Evidence
C:\Users\Lenovo\Desktop\Sperenza\Spring_2026\iot\single_project\source\results\summaries\mqtt_listener_2026-04-18_live.txt

connecting to broker 192.168.1.33:1883 and waiting for topic project/adaptive-sampling-node/aggregate
subscribed to project/adaptive-sampling-node/aggregate on 192.168.1.33:1883 as edge-listener
message window=306 profile=clean_reference fs=40.0Hz dominant=5.0Hz avg=-0.0005 anomalies=0 listener_latency_us=1231985 end_to_end_latency_us=1584319
message window=307 profile=clean_reference fs=40.0Hz dominant=5.0Hz avg=-0.0003 anomalies=0 listener_latency_us=1232218 end_to_end_latency_us=1584218
message window=308 profile=clean_reference fs=40.0Hz dominant=5.0Hz avg=-0.0001 anomalies=0 listener_latency_us=1232303 end_to_end_latency_us=1586071
```

Figure 1. Python edge listener receiving consecutive MQTT aggregate messages from the Heltec board.

11 Three-Signal Validation

The bonus-ready signal profiles were also validated through real-board MQTT runs.

11.1 Comparison Table

Profile	Avg dominant [Hz]	Avg sampling [Hz]	Avg anomaly	Avg payload [B]	Avg listener latency [us]
`clean_reference`	5.000 Hz	40.000 Hz	0.000	446.0 B	854014.8 us
`noisy_reference`	5.000 Hz	40.000 Hz	0.000	445.6 B	845563.8 us
`anomaly_stress`	5.000 Hz	40.000 Hz	2.800	444.6 B	823576.4 us

11.2 Main Observations

- all three profiles preserved `5.00 Hz` as the dominant frequency in the saved windows,
- the anomaly profile produced non-zero `anomaly_count` values as expected,
- the payload size remained almost constant because each window still produces one compact aggregate.

```
Signal Profile Comparison
C:\Users\Lenovo\Desktop\Sperenza\Spring_2026\iot\single_project\source\results\summaries\signal_profile_comparison_2026-04-18.txt

Signal profile comparison from real-board MQTT runs

profile          dominant_hz   sampling_hz   anomaly_count_avg   payload_size_avg_B   listener_latency_avg_us
clean_reference  5.000         40.000        0.000               446.0                854014.8
noisy_reference  5.000         40.000        0.000               445.6                845563.8
anomaly_stress   5.000         40.000        2.800               444.6                823576.4

source files
- mqtt_summary_2026-04-17_clean_profile_synced.md
- mqtt_summary_2026-04-17_noisy_profile_synced.md
- mqtt_summary_2026-04-17_anomaly_profile_synced.md
```

Figure 2. Comparison of the three saved signal-profile runs captured from the real board over MQTT.

Anomaly Evidence
C:\Users\Lenovo\Desktop\Sperenza\Spring_2026\iot\single_project\source\results\summaries\anomaly_evidence_2026-04-17.txt

```
Non-zero anomaly evidence from the saved anomaly-stress MQTT run

window_id   signal_profile   anomaly_count   dominant_frequency_hz   average_value
2           anomaly_stress   1               5.0                     -0.0379
3           anomaly_stress   4               5.0                      0.1560
4           anomaly_stress   5               5.0                     -0.1217
5           anomaly_stress   2               5.0                      0.0814
6           anomaly_stress   2               5.0                     -0.0065

source file
- mqtt_run_2026-04-17_anomaly_profile_synced.jsonl
```

Figure 3. Non-zero anomaly counts recorded in the anomaly-stress profile.

12 BetterSerialPlotter Validation

The project includes a simplified serial stream for `BetterSerialPlotter`. This visual evidence shows:

- current sampling frequency,
- dominant frequency,
- aggregate average.

This figure is useful for presenting the live signal-processing pipeline, but it is not used as proof of MQTT or TTN delivery.

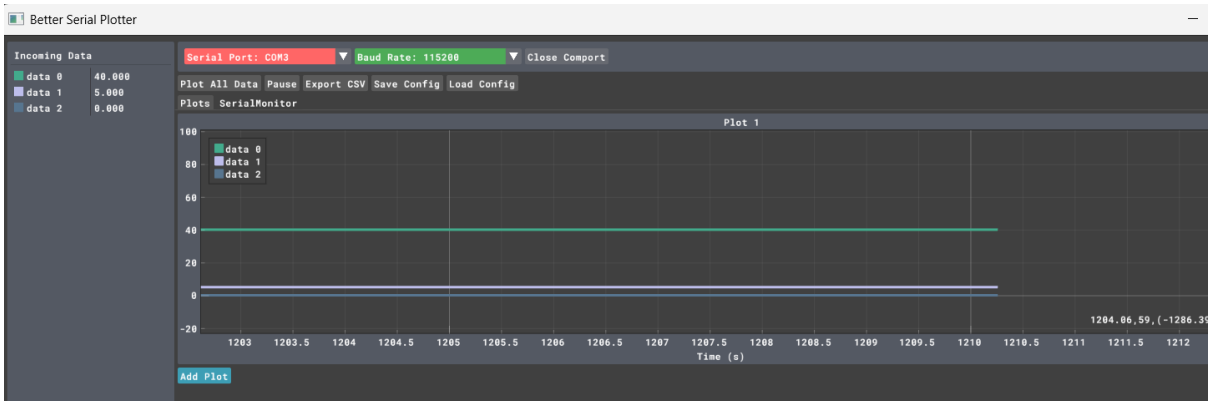

Figure 4. BetterSerialPlotter receiving the live adaptive-sampling stream from the Heltec board.

13 LoRaWAN And TTN Status

The `LoRaWAN` path is implemented up to the payload-preparation stage.

What is already true:

- the aggregate is serialized into a compact uplink payload,

- the LoRa path is integrated into the same pipeline as MQTT,
- the runtime logs show prepared payloads such as `000100C8019001F4FFE8`.

What was later validated:

- a real uplink through a reachable gateway,
- `TTN` live data screenshots,
- decoded cloud-side proof.

So the correct current description is:

- `LoRaWAN/TTN` path is implemented and validated,
- serial and `TTN` evidence are saved in the repository.

14 Secure MQTT Status

The MQTT client now supports:

- plain `mqtt://`,
- verified `mqtts://`,
- optional username/password authentication,
- certificate verification through the ESP-IDF CA bundle.

However, the captured saved run is still the plain local-broker run.
So the honest current status is:

- secure MQTT support is implemented in firmware,
- a live secure-broker validation run is still pending.

15 Energy Measurement Status

Energy was measured with the same two-board `INA219` setup described in the runbook and used by the reference-style repositories:

- one Heltec DUT running the main FreeRTOS firmware,
- one Heltec monitor reading the `INA219`,
- DUT powered only through the `INA219` high-side path,
- same WiFi network, `MQTT_ONLY` mode, and `clean_reference` profile for the comparable runs.

Baseline Versus Adaptive

Metric	Baseline 50 Hz	Adaptive 40 Hz	Delta
Average current	110.4953 mA	110.6984 mA	+0.18%
Average power	553.0000 mW	552.6775 mW	-0.06%
Integrated energy	18.433238 mWh	18.422466 mWh	-0.06%
Peak power	873.0000 mW	793.0000 mW	-9.16%

Evidence:

- `results/summaries/ina219_baseline_2026-04-21.md`,
- `results/summaries/ina219_adaptive_2026-04-21.md`,
- `results/summaries/ina219_comparison_2026-04-21.md`,
- `pics/hardware.png`,
- `pics/2026-04-21_ina219_adaptive_betterserialplotter.png`.

The honest interpretation is that adaptive sampling reduces the sample-processing rate, but WiFi, display, MQTT, and always-on FreeRTOS activity dominate this board's electrical profile. Therefore, the measured average energy improvement is small, while the peak power is lower in the adaptive run.

Optional Deep Sleep Comparison

An optional adaptive + deep-sleep experiment was also captured. It runs the adaptive pipeline, then enters timed deep sleep cycles. This reduced integrated energy to `13.632451 mWh` over `119.480 s`, about `26.04%` lower than the fixed `50 Hz` awake baseline.

Evidence:

- `results/summaries/ina219_deepsleep_2026-04-21.md`,
- `results/summaries/ina219_three_mode_comparison_2026-04-21.md`,
- `pics/2026-04-21_ina219_deepsleep_betterserialplotter.png`.

This should be presented as an additional low-power strategy rather than as the required adaptive-sampling comparison.

16 Communication Volume Discussion

One subtle but important observation is that this project sends one aggregate per completed window, not raw samples over MQTT.

That means:

- adaptive sampling reduces local sampling and processing work,
- but it does not automatically reduce the MQTT message size very much.

This is exactly what the saved payload summaries and the final communication-volume comparison show:

- payload sizes remain around `444` to `454` bytes depending on the profile.
- fixed `50 Hz` represented `1250` samples across five aggregate windows,
- adaptive `40 Hz` represented `1000` samples across five aggregate windows,
- both modes sent five aggregate `MQTT` messages, so total aggregate payload stayed `2270 B`.

So the communication-volume section in the final report should explain that:

- the main gain from adaptive sampling in this implementation is local efficiency,
- the network payload count per window is mostly unchanged.

Evidence: `results/summaries/communication_volume_comparison_2026-04-21.md`.

17 Build And Resource Status

The current firmware still builds successfully after the recent additions such as:

- three signal profiles,
- secure MQTT support,
- baseline/adaptive configuration switch.

Current build snapshot:

Resource	Usage
RAM	`38120 B` (`11.6%`)
Flash	`827875 B` (`24.8%`)

This leaves comfortable flash headroom for the workshop build.

18 Remaining Work

The remaining work before final submission is:

Item	Status	Notes
`LoRaWAN/TTN` live proof	complete	serial and TTN screenshots are saved from the integrated main app
Energy measurements	complete	INA219 baseline/adaptive runs are saved and compared
Secure MQTT live TLS proof	pending	firmware support exists, but no saved live TLS run yet
Final short report	complete	submission snapshot and evidence matrix now capture the current state

19 Conclusion

At this checkpoint, the project has moved well beyond the early scaffolding phases.

The most important completed achievements are:

- stable virtual-signal generation on the real board,
- measured maximum stable sampling benchmark,
- correct dominant-frequency detection,
- working adaptive-sampling transition from `50 Hz` to `40 Hz`,
- correct per-window aggregate computation,
- full home-network `MQTT/WiFi` validation on hardware,
- real latency measurements based on synchronized timestamps,
- three validated signal profiles including anomaly-aware evidence,
- completed communication-volume comparison,
- completed Z-score and Hampel anomaly-filter evaluation.

The most important missing items are now concentrated and clear:

- one secure `MQTTS` validation run.

This means the project is already in a strong state for the workshop: the core pipeline works, the edge and cloud paths are proven, and the remaining work is focused on optional secure-MQTT proof rather than on missing core functionality.